# SC2001

# PROJECT 1

**TEAM 7**

**Nguyen An Loc**

**Ooi Jia xi**

**Panicker Shobith Shreekumar**

# PROJECT 1: INTEGRATION OF MERGESORT AND INSERTION SORT

**1. Algorithm implementation**
Implement and analyze the hybrid algorithm

**2. Generate input data**
Generate arrays of increasing sizes, in a range from 1,000 to 10 million

**3. Analyze Time complexity**
Determine optimal value of S

**4.Compare with Mergesort**
Compare performance with the original Mergesort in lecture

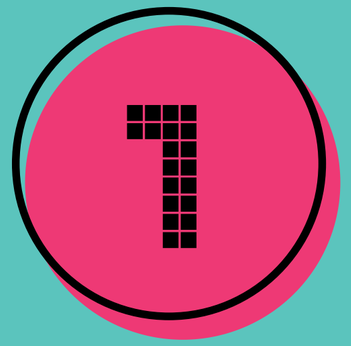
# Motivation for Hybrid Sort

- MergeSort efficient overall but has recursion overhead on small subarrays
- Insertion Sort is simple and efficient for small subarrays
- Hybrid approach: combine both for better performance

→

- Use MergeSort for large subarrays
- Switch to Insertion Sort when subarray size ≤ threshold S
- Balances divide-and-conquer with localized efficiency

# Part (a): Algorithm implementation

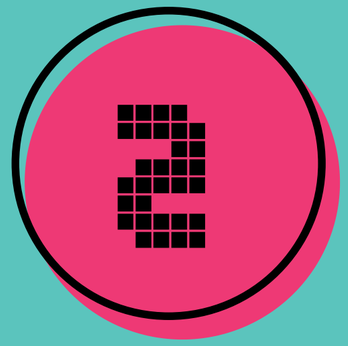


## Insertion Sort

```python
def insertionSort(arr, keyC, left, right):
    '''
    Arguments:
        arr (np.array): array to be sorted
        keyC (int)    : number of key comparisons so far
        left (int)    : starting index of the subarray
        right (int)   : ending index of the subarray (NOT the array size)

    Return:
        keyC  (int)   : number of key comparisons after sorting
    '''

    # Trivial case: If subarray has 0 or 1 element, nothing to sort
    if left >= right:
        return keyC

    # Outer loop: Traverse each element in the subarray
    for i in range(left, right + 1):
        # Inner loop: Compare current element backwards with previous elements
        for j in range(i, left, -1):
            if arr[j] < arr[j - 1]:
                keyC += 1
                swap(arr, j, j - 1)
            else:
                keyC += 1
                break                    # stop when correct position is found

    return keyC
```

# Part (a): Algorithm implementation

```python
def merge(arr, keyC, left, mid, right):
    """
    Merge two sorted subarrays [left, ... ,mid] & [mid+1, ... ,right]
    Count and return keyC
    """
    # Trivial case
    if left >= right:
        return keyC
    # Size of the sorted array
    sorted_size = right - left + 1
    # Initialize temp array of length sorted_size
    sorted_arr = np.zeros(sorted_size, dtype = int)
    # Pointer index for left and right subarrays
    idex1 = left
    idex2 = mid + 1
    # Pointer index for temp array
    i = 0
    loop = True
    while loop:
        if arr[idex1] < arr[idex2]:
            # Increase keyC by 1 after every comparison
            keyC += 1
            # If the value of the arr1 at idex1 is smaller, insert it into temp array
            sorted_arr[i] = arr[idex1]
            # Increase the temp array's pointer index to the empty slot
            i += 1
            # If the arr1's pointer hit the end (mid), but arr2's pointer have not reached the end
            # Loop and insert all remaining elements in arr2
            if idex1 == mid:
                while(idex2 <= right):
                    sorted_arr[i] = arr[idex2]
                    i += 1
                    idex2 += 1
                # After inserted all the elements in both array into one array, we break the main loop
                break
            # If the arr1's pointer have not reached the end, keep increasing its pointer
            else:
                idex1 += 1
            else:
                # Same as the above code but for arr2
                keyC += 1
                sorted_arr[i] = arr[idex2]
                i += 1
                if idex2 == right:
                    while(idex1 <= mid):
                        sorted_arr[i] = arr[idex1]
                        i += 1
                        idex1 += 1
                    break
                else:
                    idex2 += 1
    # Copy temp merged array back into the original array with corresponding positions
    arr[left:right + 1] = sorted_arr.tolist()
    return keyC
```



**Merge the two sorted subarrays**

# Part (a): Algorithm implementation

## 3 Hybrid Sort

## 4 Testing

This **hybridSort()** adds an extra argument from the original **mergeSort()**, allow us to set an 'S' value.
When S = 0, the algorithm behaves exactly like the **mergeSort()** taught in lectures.

```python
def hybridSort(arr, keyC, left, right, S = 0):
    # Threshold base case: When subarray size is smaller than S
    # => use Insertion Sort
    if (right - left + 1) <= S:
        keyC = insertionSort(arr, keyC, left, right)
        return keyC

    if left >= right:
        return keyC

    # Recursively sort each half
    else:
        mid = (left + right) // 2
        keyC = hybridSort(arr, keyC, left, mid, S)
        keyC = hybridSort(arr, keyC, mid + 1, right, S)

        # Merge the two sorted halves
        keyC = merge(arr, keyC, left, mid, right)

        return keyC
```

```python
arr = [42, 15, 23, 4, 16, 8, 55, 0, 19, 31]
print("Original array:", arr)
keyC = 0

# Run Hybrid Sort with threshold S = 2
keyC = hybridSort(arr, keyC, 0, len(arr) - 1, S = 2)
print("Sorted array:", arr)
print("Number of key comparisons:", keyC)
```

✓ 0.0s

```
Original array: [42, 15, 23, 4, 16, 8, 55, 0, 19, 31]
Sorted array: [0, 4, 8, 15, 16, 19, 23, 31, 42, 55]
Number of key comparisons: 23
```

# (b) Generate input data

## 1. Define dataset sizes

```
# Initialize an array of dataset sizes
dataset_sizes = []

# Create sizes in steps of 1k, 10k, 100k, and 1M
for k in range(10):
    dataset_sizes.append((k+1) * 1_000)
    dataset_sizes.append((k+1) * 10_000)
    dataset_sizes.append((k+1) * 100_000)
    dataset_sizes.append((k+1) * 1_000_000)

# Ensure no duplicates and keep the list sorted
dataset_sizes = sorted(set(dataset_sizes))
print(dataset_sizes)
```

✓ 0.0s

```
[1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000, 9000, 10000,
```

## 2. Generate the datasets

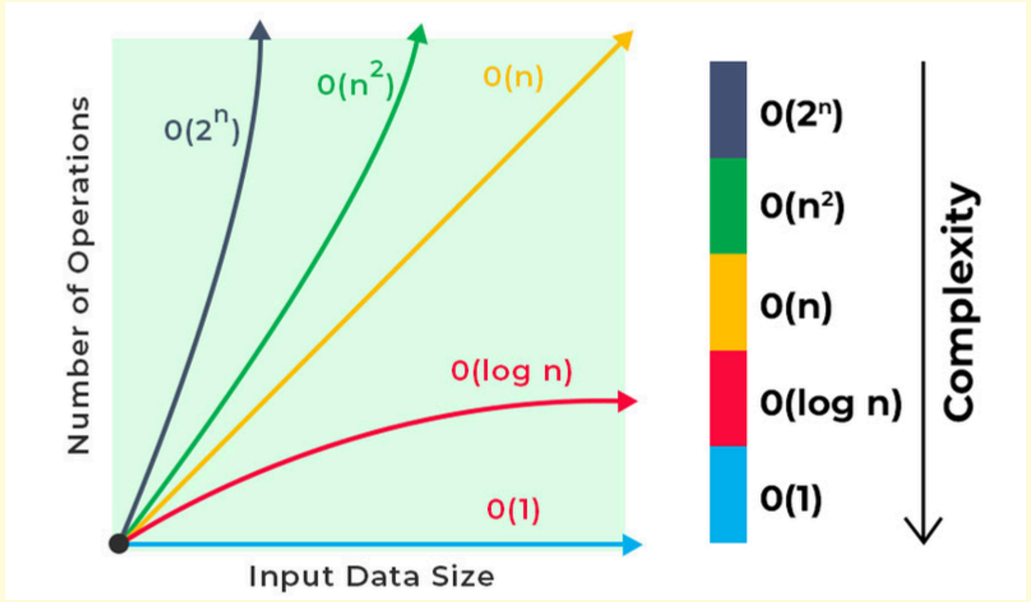
```
# List of List of data
inputData = []
for size in dataset_sizes:
    # For each datasize, generate a random data array of length size
    # Each array will contain random integers between 1 to s.
    data = np.random.randint(1, size+1, size = size)
    inputData.append(data)

for i in range(len(inputData)):
    print(f"Array {i+1}: Size = {len(inputData[i])}")
    print("Min value: " , min(inputData[i]))
    print("Max value: " , max(inputData[i]))
    print()
```

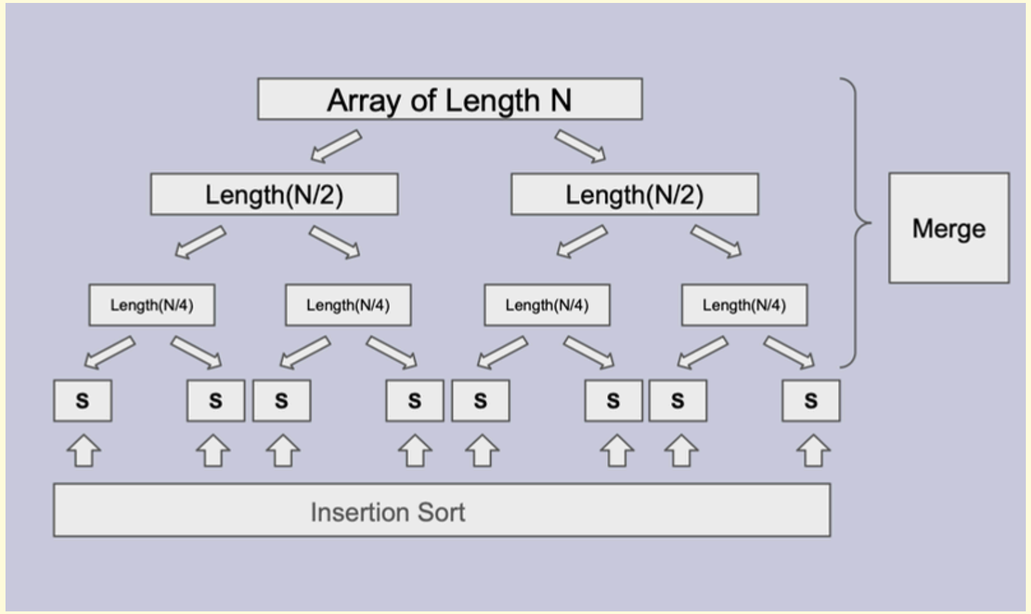
✓ 4.2s

# Part (c): Analysis of time complexity

## Theoretical time complexity

| Algorithm | Best Case | Worst Case | Average Case |
|---|---|---|---|
| MergeSort | O(n log n) | O(n log n) | O(n log n) |
| Insertion Sort | O(n) | $O(n^2)$ | $O(n^2)$ |
| Hybrid Sort | O(n log(n/S)+ n) | O(n log(n/S)+ nS) | O(n log(n/S)+ nS) |




- n: the number of elements in the array you're sorting (input size).
- S: the cutoff threshold at which the algorithm stops recursing and switches from mergesort to insertion sort for any subarray of size ≤S.

# Part (c): Analysis of time complexity

## Theoretical time complexity

### Theoretical Analysis

#### Insertion Sort

When using the hybrid algorithm and we use **Insertion Sort** when subarray sizes become $\leq S$, the average time complexity is equal to the worst time complexity, which is $O(s^2)$, where the best case is $O(s)$.

Hence, the **worst-case time complexity** for the Insertion Sort part of the algorithm is:

$$\frac{n}{S} \times O(s^2) = O(nS)$$

where $\frac{n}{S}$ is the number of subarrays whose size is $\leq S$.

The **best-case time complexity** is:

$$\frac{n}{S} \times O(s) = O(n)$$

---

#### Merge Sort

For regular **Merge Sort**, the problem size is repeatedly halved at each level, and we stop when we reach a problem size of 1, resulting in $\log_2 n$ levels.

For a **Merge Sort** that switches to **Insertion Sort** when subarrays have size $\leq S$, we end up with the number of merge levels being $\log_2\left(\frac{n}{S}\right)$. Since, at each level, the total size of all subproblems is still n, merging across that entire level costs $O(n)$. Hence overall time complexity for **Merge Sort** is:

$$O\left(n \log_2\left(\frac{n}{S}\right)\right)$$

---

#### Overall Time Complexity

Thus, the overall time complexity is:

$$O\left(n \log_2\left(\frac{n}{S}\right) + nS\right)$$

- **n: the number of elements in the array you're sorting (input size)**
- **S: the cutoff threshold at which the algorithm stops recursing & switches from mergesort to insertion sort for any subarray of size ≤ S.**

# Part (c): Analysis of time complexity

## Fix S vary n (Theoretical)

**Mergesort performance (assume $n = 2^k$)**

**Worst case :**

$W(1) = 0,$

$W(n) = W(n/2) + W(n/2) + n-1$ Or

$W(2^k) = 2W(2^{k-1}) + 2^k - 1$

$= 2(2W(2^{k-2}) + 2^{k-1} - 1) + 2^k - 1$

$= 2^2W(2^{k-2}) + 2^k - 2 + 2^k - 1$

$= 2^2(2W(2^{k-3}) + 2^{k-2} - 1) + 2^k - 2 + 2^k - 1$

$= 2^3W(2^{k-3}) + 2^k - 2^2 + 2^k - 2 + 2^k - 1$

…

$= 2^kW(2^{k-k}) + k2^k - (1 + 2 + 4 + \ldots + 2^{k-1})$

$= k2^k - (2^k - 1)$

$= n \lg n - (n - 1)$

$= O(n \lg n)$

$k = \lg n$

Geometric series

Base Case(Insertion Sort starts at S)

$$W(S) = \frac{(S-1)(S+2)}{4} \approx \frac{S^2}{4}$$

Let $n = 2^k$, $S = 2^j$

$\frac{n}{S} = \frac{2^k}{2^j} = num.\ of\ size\ S\ subarrays$

$$W(2^k) = 2 \cdot W\left(\frac{2^k}{2}\right) + 2^k - 1$$

$$= 2\ \left[2 \cdot W\left(\frac{2^{k-1}}{2}\right) + 2^{k-1} - 1\right] + 2^k - 1$$

$$= 2^2 \cdot W\left(2^{k-2}\right) + 2^k - 2 + 2^k - 1$$

$$= 2^2 \cdot W\left(2^{k-2}\right) + 2 \cdot 2^k - [2+1]$$

$$\ldots$$

$$= 2^{k-j} \cdot W\left(2^{k-(k-j)}\right) + 2^k \cdot (k-j) - \left[2^{k-j} - 1\right]$$

$$= \frac{n}{S} \cdot W(S) + n \cdot (k-j) - \left[\frac{n}{S} - 1\right]$$

$$= \frac{n}{S} \cdot W(S) + n\ \ (\log_2 n - \log_2 S) - \left[\frac{n}{S} - 1\right]$$

$$= \frac{n}{S} \cdot W(S) + n \cdot \left(\log_2 \frac{n}{S}\right) - \left[\frac{n}{S} - 1\right]$$

$$= \frac{n}{S} \cdot \frac{S^2}{4} + n \cdot \left(\log_2 \frac{n}{S}\right) - \left[\frac{n}{S} - 1\right] \qquad = O\left(nS + n \cdot \log\left(\frac{n}{S}\right)\right)$$

# Part c(i): Analysis of time complexity

## Fix S vary n (Code)

```python
def experiment_ci_with_existing_hybridSort(inputData, S, progress_every=5):
    """
    Runs hybridSort on each pre-generated dataset in inputData (once each).
    progress_every: print progress every this many datasets just for clarity.
    Returns (ns, comps).
    """
    ns, comps = [], []
    total = len(inputData)
    for i, data in enumerate(inputData, start=1):
        n = len(data)

        arr = data.copy()
        keyC_after = hybridSort(arr, 0, 0, n - 1, S=S)
        ns.append(n)
        comps.append(keyC_after)

        # Print comparisons in the required format
        print(f"Current Array Size: {n}")
        print(f"Number of Key Comparisons: {keyC_after}")
        print()

        if progress_every and (i % progress_every == 0 or i == total):
            print(f"{i}/{total} processed — n={n}, comparisons={keyC_after}")
    return ns, comps

#Run (c)(i)
S_fixed = 15
ns, comps = experiment_ci_with_existing_hybridSort(inputData, S_fixed, progress_every=5)
```

### Overall Time Complexity

Thus, the overall time complexity is:

$$O\left(n \log_2\left(\frac{n}{S}\right) + nS\right)$$

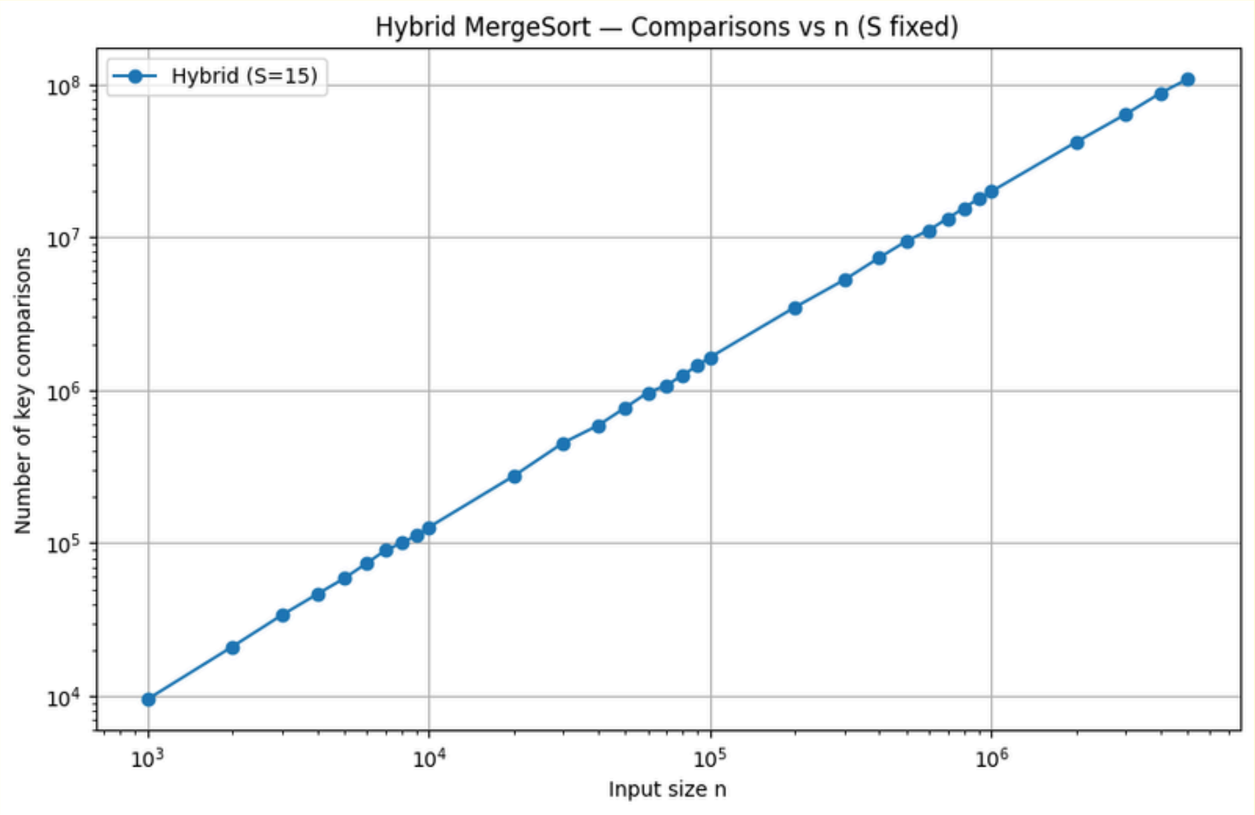
With $S$ **fixed** (a constant), this simplifies to:

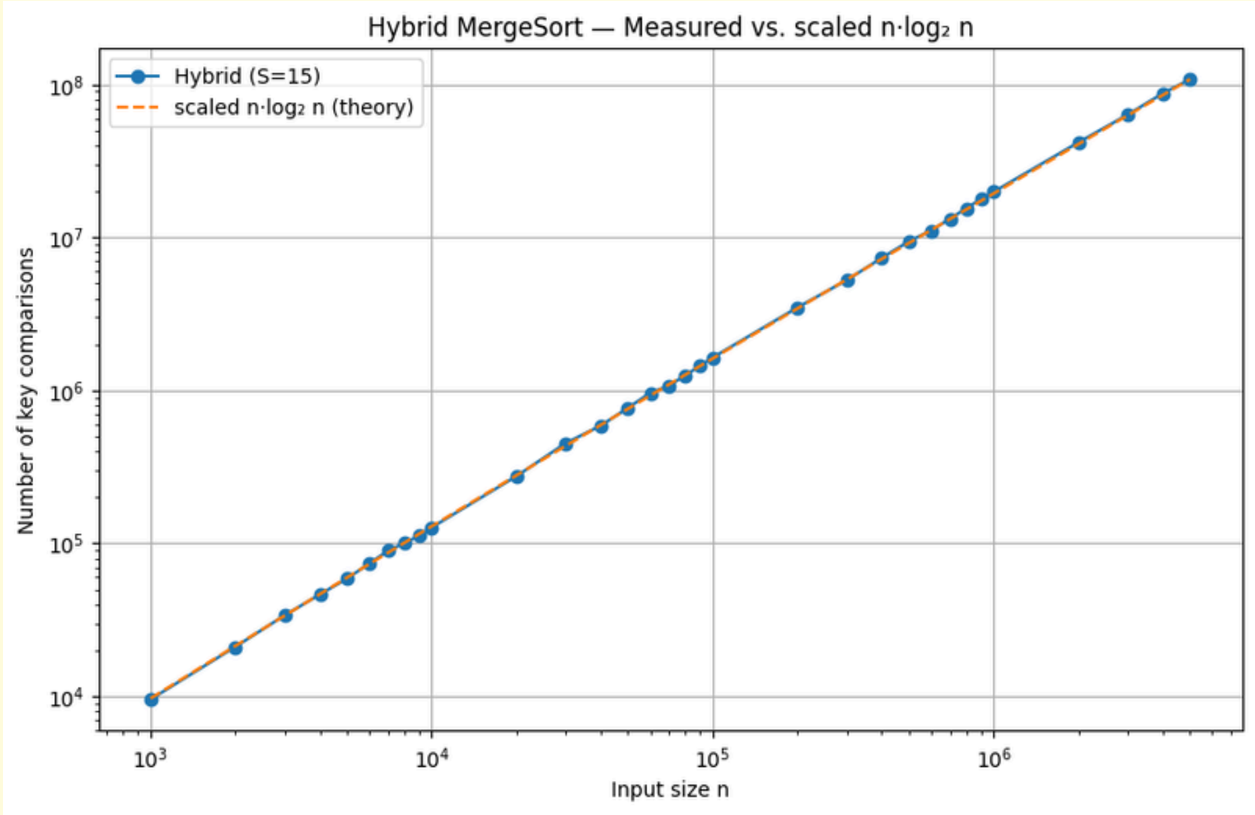
$$O(n \log n)$$

So, the number of comparisons should grow on the order of $n \log n$.

# Part c(i): Analysis of time complexity

## Fix S vary n




```
Current Array Size: 1000
Number of Key Comparisons: 9557

Current Array Size: 2000
Number of Key Comparisons: 21047

Current Array Size: 3000
Number of Key Comparisons: 33860

Current Array Size: 4000
Number of Key Comparisons: 46320

Current Array Size: 5000
Number of Key Comparisons: 58837
```

# Part (c)ii: Analysis of time complexity

## Rationale for choice of thresholds (S values)

- **For very small S (2, 4):**
  - Almost always using MergeSort.
  - High recursion overhead.
  - Runtime larger.

- **For medium S (8, 16, 32):**
  - Balanced use of MergeSort + InsertionSort.
  - Runtime minimized around here (usually between 8 and 32).

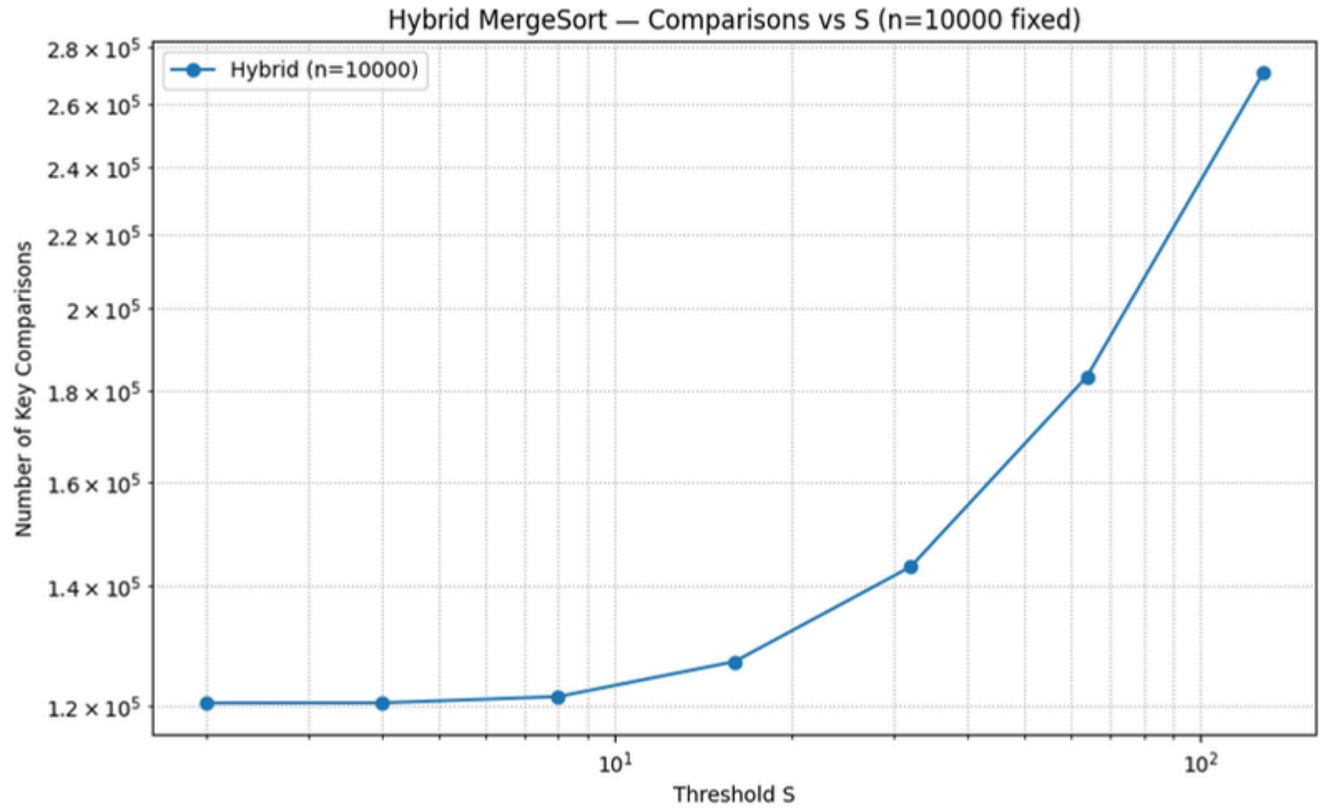
- **For large S (64, 128):**
  - Large subarrays handled by InsertionSort.
  - Runtime increases rapidly because InsertionSort is O(n^2).

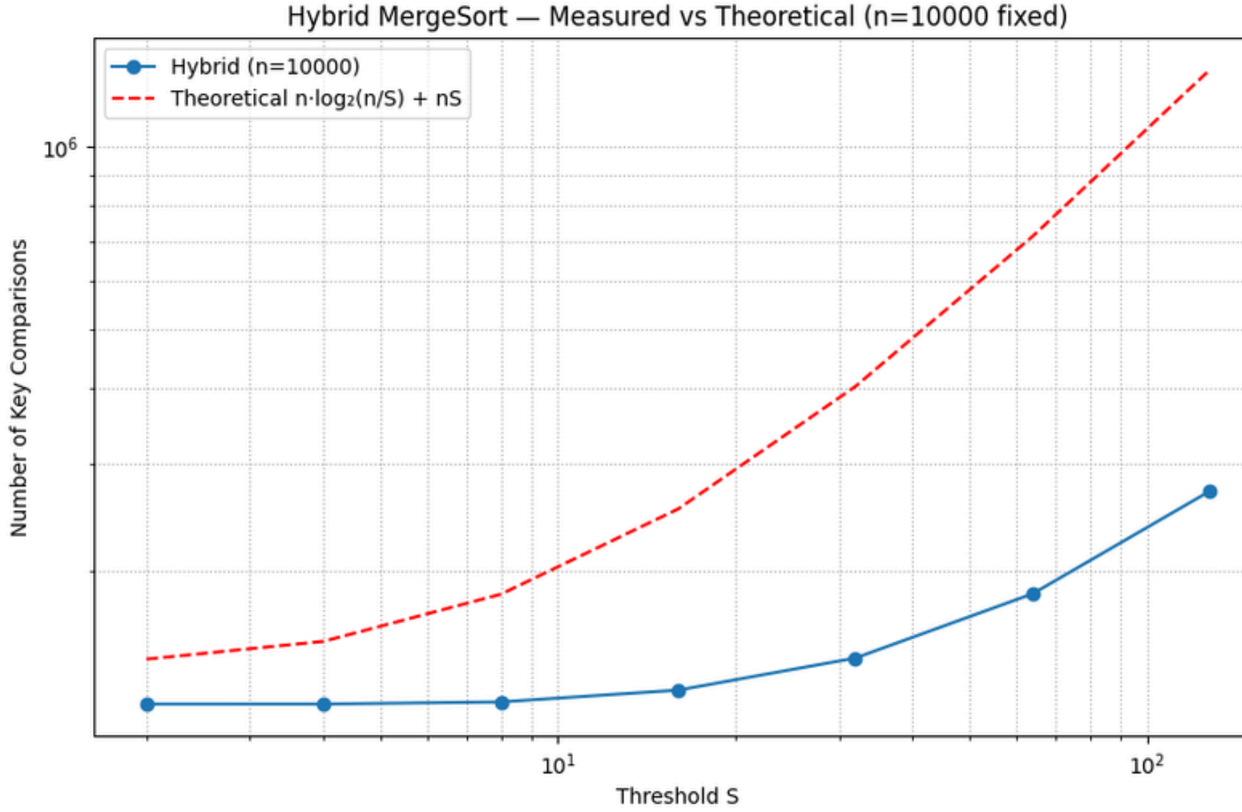
So the rationale is: By choosing exponentially increasing S values, we cover the whole range of possible trade-offs between recursion overhead and insertion overhead.

# Part (c)ii

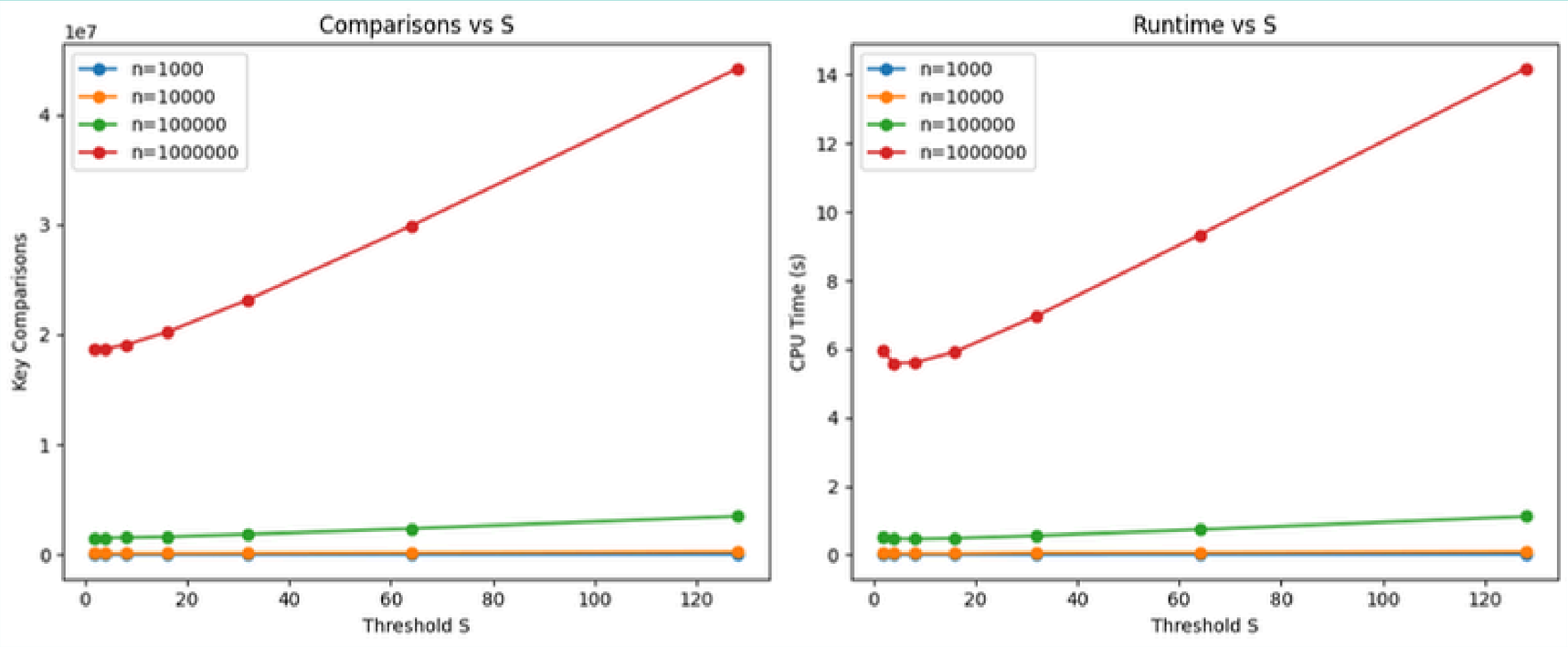
## Fix N vary S

```
S Value: 2, Key Comparisons: 120427 Time taken: 0.172858 seconds
S Value: 4, Key Comparisons: 120449 Time taken: 0.168377 seconds
S Value: 8, Key Comparisons: 121427 Time taken: 0.158563 seconds
S Value: 16, Key Comparisons: 127007 Time taken: 0.162375 seconds
S Value: 32, Key Comparisons: 143481 Time taken: 0.179317 seconds
S Value: 64, Key Comparisons: 183210 Time taken: 0.246472 seconds
S Value: 128, Key Comparisons: 270698 Time taken: 0.399395 seconds
```



# Part (c)iii : Finding optimal S



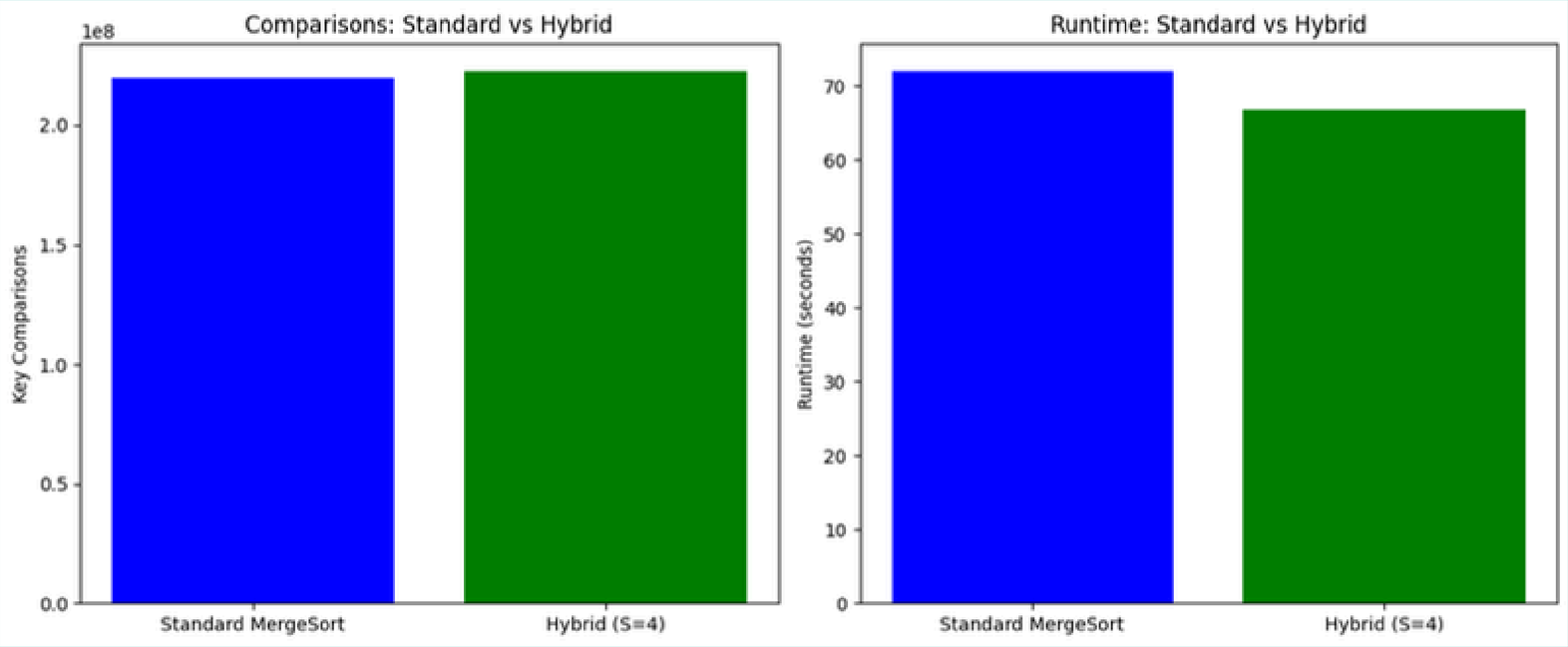

**Comparisons ↑ as S increases (insertion sort dominates).**

**Runtime forms U-shape.**

**(small S lead to recursion overhead, large S makes insertion sort too slow)**

**Lowest runtime at S = 4.**

# Part (d) : Standard Mergesort vs. Hybrid Sort




## Results

The table below shows the performance comparison:

| Algorithm | Key Comparisons | Runtime (s) |
|---|---|---|
| Standard Mergesort | *220100135* | *72.0582* |
| Hybrid Sort (S = 4) | *222895442* | *66.7621* |

**Using dataset with 10 million integers and S = 4, Hybrid Sort shows similar number of comparisons to standard Mergesort but shorter runtime.**


THANK YOU!